Next-Day Stock Price Prediction with PCA + MLP

MarcAnthony Cantu

April 2026

Data Preparation

Each of the eight stock files was loaded and the three columns *Open*, *High*, and *Low* were dropped. The remaining rows were filtered to keep only those with timestamp between January 1, 2010 and March 31, 2023, and the DOW index file was discarded.

From the eight remaining stocks, four were selected at random:

$$S_1 = \text{GE}, \quad S_2 = \text{JPM}, \quad S_3 = \text{MSFT}, \quad S_4 = \text{RTX}.$$

For each S_j a table T_j with columns *Date*, *Time*, *Close*, *Volume* was built by splitting the original *Date&Time* string on whitespace.

Aligning trading days. The set $L^{(j)}$ of distinct trading days in T_j was extracted for each of the four stocks, and the four tables were filtered to their common intersection

$$L = L^{(1)} \cap L^{(2)} \cap L^{(3)} \cap L^{(4)} = \{D_1, D_2, \ldots, D_N\}.$$

After this step the four tables share exactly the same set of N trading days.

Per-Day Feature Vectors

For each day D_n and stock S_j, the intraday close prices form a sequence $P_j(n) = \{p_1^{(n)}, \ldots, p_r^{(n)}\}$ and the volumes form $V_j(n) = \{v_1^{(n)}, \ldots, v_r^{(n)}\}$, where the length r varies from day to day and stock to stock. Ten summary statistics were computed:

$$\begin{aligned}
&\text{pinit}_j(n) = p_1^{(n)}, && \text{plast}_j(n) = p_r^{(n)}, && \text{pmed}_j(n) = \text{median}(P_j(n)),\\
&\text{plow}_j(n) = Q_{0.05}(P_j(n)), && \text{phigh}_j(n) = Q_{0.95}(P_j(n)), && \text{vinit}_j(n) = v_1^{(n)},\\
&\text{vlast}_j(n) = v_r^{(n)}, && \text{vmed}_j(n) = \text{median}(V_j(n)), && \text{vlow}_j(n) = Q_{0.05}(V_j(n)),\\
&\text{vhigh}_j(n) = Q_{0.95}(V_j(n)).
\end{aligned}$$

These statistics were grouped into the two row vectors

$$pU_j(n) = [\text{pinit}_j, \text{plast}_j, \text{pmed}_j, \text{plow}_j, \text{phigh}_j],$$

$$vU_j(n) = [\text{vinit}_j, \text{vlast}_j, \text{vmed}_j, \text{vlow}_j, \text{vhigh}_j],$$

and the eight resulting row vectors were concatenated into the 40-dimensional daily summary

$$W(n) = [pU_1(n), pU_2(n), pU_3(n), pU_4(n), vU_1(n), vU_2(n), vU_3(n), vU_4(n)] \in \mathbb{R}^{40}.$$

Feature Matrix X and Target Vector Y

The feature vector available on day D_n concatenates the daily summaries of the past ten trading days:

$$X(n) = [W(n), W(n-1), \ldots, W(n-9)] \in \mathbb{R}^{400}.$$

The target is the vector of next-day closing prices:

$$Y_n = [\text{plast}_1(n+1), \text{plast}_2(n+1), \text{plast}_3(n+1), \text{plast}_4(n+1)] \in \mathbb{R}^4.$$

Because $X(n)$ requires nine previous days and Y_n requires the next day, the admissible index range is $n = 9, 10, \ldots, N-2$, giving an effective sample size of $N_{\text{eff}} = N - 10$.

Principal Component Analysis

Each of the 400 coordinates of X was standardized independently:

$$H_{n,i} = \frac{X_{n,i} - \mu_i}{\sigma_i}, \qquad i = 1, \ldots, 400,$$

where $\vec{\mu}$ and $\vec{\sigma}$ are the per-coordinate means and standard deviations across all n. The correlation matrix $\text{Corr}(H)$ was then computed, and its eigenvalues and eigenvectors were extracted and sorted in descending order

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_{400}.$$

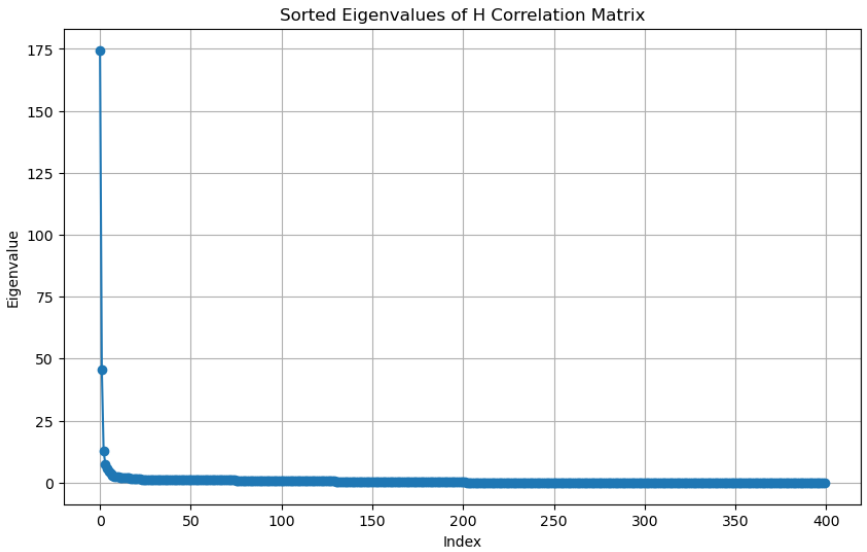


Figure 1: Sorted Eigenvalues

The percentage of variance explained by the first r components is

$$\text{PEV}(r) = \frac{\sum_{i=1}^{r} \lambda_i}{\sum_{i=1}^{400} \lambda_i}.$$

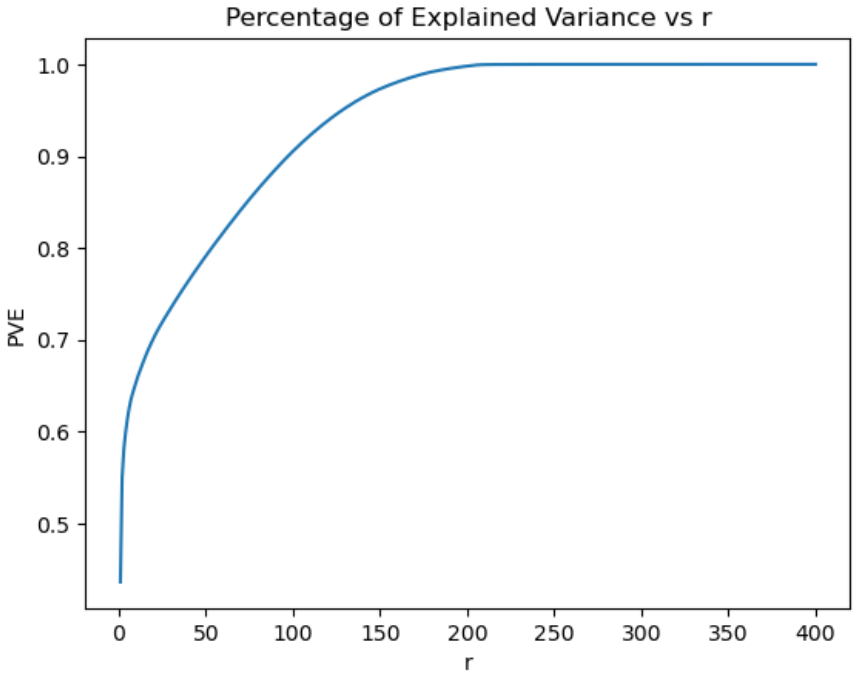


Figure 2: Percentage of variance explained

The smallest r such that $\mathrm{PEV}(r) \approx 95\%$ was found to be

$$r = 128,$$

The reduced features are then

$$H_r = H\,W_r \in \mathbb{R}^{128},$$

where W_r is the 400×128 matrix whose columns are the top-r eigenvectors.

Splitting the Dataset

The MLP architecture is three layers:

$$L_1 \text{ (input)} \longrightarrow L_2 \text{ (hidden)} \longrightarrow L_3 \text{ (output)},$$

with $\dim(L_1) = r = 128$ and $\dim(L_3) = 4$. The size $h = \dim(L_2)$ is chosen in Q6.

The N_{eff} cases were partitioned at random into a training set (85%) and a test set (15%) using `sklearn.model_selection.train_test_split` with `random_state=42`. The targets Y_n were also standardized using the training-set mean $\vec{\mu}_Y$ and standard deviation $\vec{\sigma}_Y$; the same affine transformation was then applied to the test targets, keeping both sets on a common scale for MSE evaluation. The loss function is

$$\mathrm{MSE} = \frac{1}{|\text{train.set}|} \sum_{n \in \text{train.set}} \|Y_n - Z_n\|^2.$$

Finding the optimal layer size

For a 3-layer fully-connected MLP with layer sizes $[k, h, 4]$, the number of trainable parameters is

$$\mathrm{PAR}(h) = (r+1)\,h + 5\,h + 4 = (r+5)\,h + 4.$$

Each training case contributes one target vector $Y_n \in \mathbb{R}^4$ with four scalar entries, so the total amount of information available from the training labels is

$$M = 4 \times |\text{train.set}|.$$

We searched for the integer g that makes $\mathrm{PAR}(g)/M$ closest to 1. With $r = 128$ this yields

$$g = 84, \qquad \mathrm{PAR}(g)/M \approx 1.0007.$$

Doubling the hidden layer to $h = 2g = 168$ gives

$$\mathrm{PAR}(2g)/M \approx 2.0011,$$

Training MLP$(2g)$

The MLP with hidden-layer size $h = 2g = 168$ was trained using `sklearn.neural_network.MLPRegressor` with random weight initialization, batch size 50, 250 epochs, MSE loss, and the Adam optimizer. Each epoch was executed by calling `partial_fit` once on the training set; the training RMSE, test RMSE, and the Euclidean norm of the per-epoch weight update were recorded.

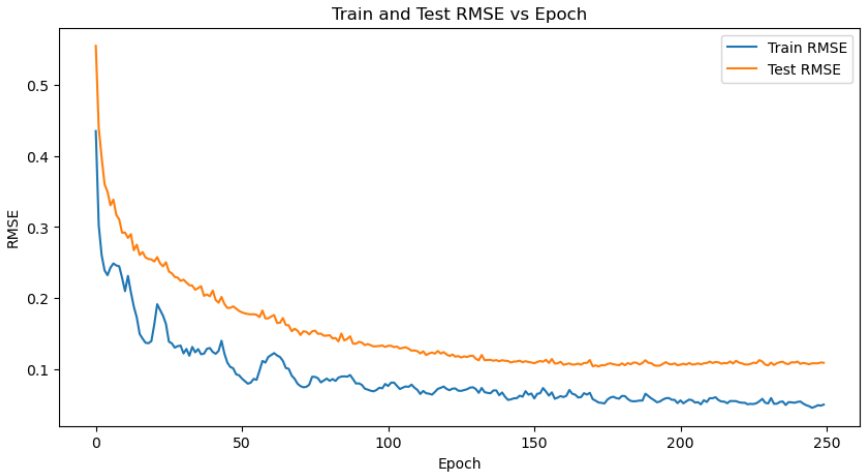


Figure 3: RMSE vs Epoch

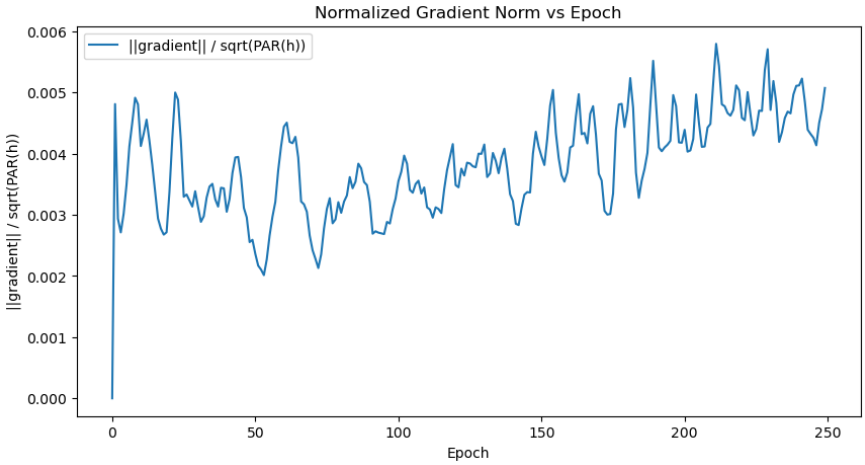


Figure 4: Normalized gradient vs Epoch

Optimal Epoch and Performance of MLP$(2g)$

The training RMSE drops sharply during the first roughly 20 epochs, from about 0.45 down to 0.15, after which it continues to decrease more slowly, reaching about 0.05 by epoch 250. The test RMSE drops monotonically from about 0.55 down to roughly 0.11 by the end of training, without a visible upward turn within the 250-epoch window. The two curves do not exhibit signs of overfitting on the test side. Additional epochs continue to help on the test set, but the marginal gains are very small in the second half of training.

The normalized gradient $\|\nabla \mathrm{MSE}(m)\|/\sqrt{\mathrm{PAR}(h)}$ oscillates between roughly 0.002 and 0.006 throughout training, with a slight upward drift over the second half of the run.

The optimal stopping point is taken as the epoch that minimizes the test RMSE:

$$m^* = 173.$$

Re-training the network for exactly m^* epochs gives the model MLP$(2g)$, whose final performance is

$$\mathrm{trainRMSE}(m^*) = 0.0961, \qquad \mathrm{testRMSE}(m^*) = 0.1695.$$

Both RMSE values are on the standardized target scale, so a test RMSE of 0.17 means the prediction error is, on average, about 17% of one standard deviation of next-day closing prices on the training set. The generalization gap $0.1695 - 0.0961 = 0.0734$ is moderate and consistent with the $\mathrm{PAR}/M \approx 2$ regime: the network has enough capacity to memorize part of the training data but generalizes reasonably well.

Per-Stock Relative Errors

For each stock S_j and each day D_n, the relative prediction error was computed as

$$\mathrm{RelErr}_n(S_j) = \frac{|\mathrm{plast}_j(n) - Z_n(j)|}{\mathrm{plast}_j(n)}.$$

Since the network is trained on standardized targets, the predictions and the true targets were both inverted to the raw price scale using $\vec{\mu}_Y$ and $\vec{\sigma}_Y$ before applying the formula above.

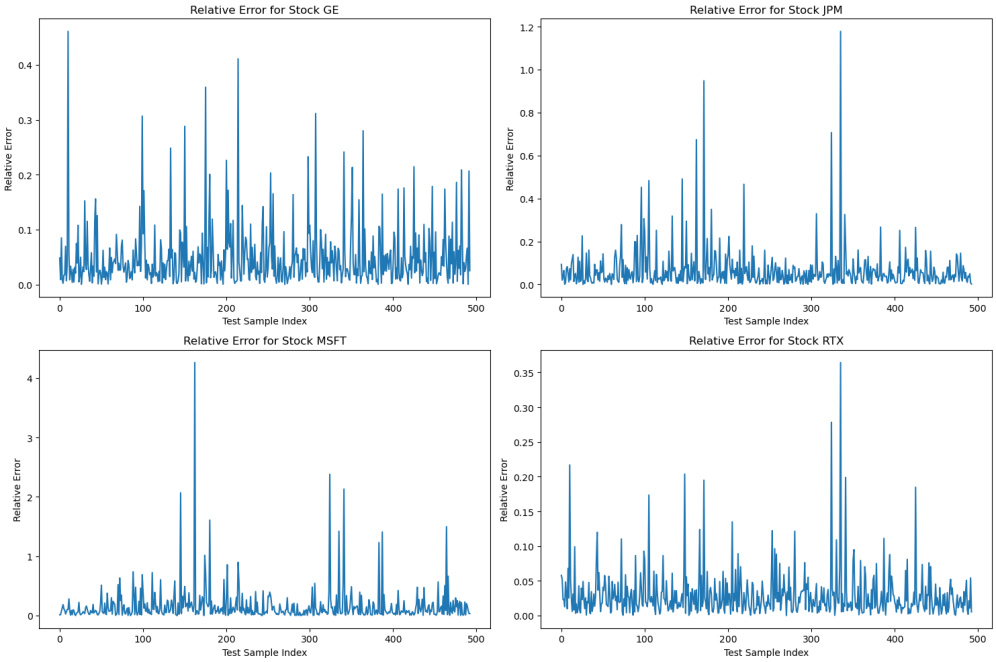


Figure 5: Relative errors of each stock

The averages of $\mathrm{RelErr}_n(S_j)$ over the training and test sets are given in Table 1.

Table 1: Average relative error of next-day closing-price prediction for each stock, on the training and test sets.

Stock	Train avg. RelErr	Test avg. RelErr
GE	0.0265	0.0486
JPM	0.0347	0.0655
MSFT	0.0686	0.1629
RTX	0.0168	0.0297

The four per-stock curves share a common shape: a low baseline of a few percent with occasional sharp spikes on individual days. GE has a baseline near 0.05 with spikes that reach 0.30–0.45; JPM is similar on average but has a few large outliers (one spike near 1.2 on a single test sample); RTX has the cleanest curve, with a baseline around 0.03 and rare spikes below 0.4. MSFT is qualitatively different: while its baseline is similar to the others (near 0.10), it has several extreme outliers in the 2–4 range, and these outliers dominate its mean. As a result the MSFT test-set average of 0.163 overstates the typical day-to-day prediction quality, which is closer to the single-digit-percent level seen for the other stocks.

Comparison with Q8. The standardized test RMSE of 0.17 is consistent with per-stock relative errors at the single-digit-percent level on most days. The two metrics agree qualitatively,

but the per-stock curves expose information that the aggregate RMSE conceals: the model is well-behaved on the vast majority of test days, and the bulk of its test error comes from a small number of high-relative-error spikes, concentrated mostly on MSFT.

A Wider Network: MLP$(5g)$

A second three-layer MLP was trained with hidden-layer size $5g = 420$, all other hyperparameters identical to Q7 and stopping again at $m^* = 173$ epochs. The performance is

$$\text{trainRMSE}(\text{MLP}(5g)) = 0.0877, \qquad \text{testRMSE}(\text{MLP}(5g)) = 0.1299.$$

Compared with MLP$(2g)$, the wider network achieves both a lower training RMSE ($0.0961 \to 0.0877$) and a noticeably lower test RMSE ($0.1695 \to 0.1299$, a 23% reduction). With $\text{PAR}(5g)/M \approx 5$ the network is substantially more over-parameterized, but early stopping at m^* prevents it from drifting into a worse generalization regime; the wider hidden layer translates here into a real improvement in test performance.

A Deeper Network: MLP(g, g)

A four-layer MLP with two hidden layers of size $g = 84$ was trained next:

$$L_1 \to L_2 \to L_3 \to L_4, \qquad \dim(L_1) = 128,\ \dim(L_2) = 84,\ \dim(L_3) = 84,\ \dim(L_4) = 4,$$

again for $m^* = 173$ epochs with all other hyperparameters unchanged. The performance is

$$\text{trainRMSE}(\text{MLP}(g, g)) = 0.0846, \qquad \text{testRMSE}(\text{MLP}(g, g)) = 0.1730.$$

The three architectures are summarized in Table 2.

Table 2: Comparison of the three MLP architectures at $m^* = 173$ epochs.

Architecture	Hidden layers	Train RMSE	Test RMSE
MLP$(2g)$	(168)	0.0961	0.1695
MLP$(5g)$	(420)	0.0877	**0.1299**
MLP(g, g)	$(84, 84)$	**0.0846**	0.1730

The deeper network MLP(g, g) has the *lowest* training RMSE of the three models but the *highest* test RMSE, a textbook sign of overfitting. Stacking two hidden layers of moderate width gives the network more representational capacity than a single wide hidden layer of equivalent parameter count, and the additional capacity is spent fitting the training set rather than generalizing.

The wider single-hidden-layer network MLP$(5g)$ is the best of the three on the test set, with test RMSE 0.1299. At this stopping point, increasing width pays off more than increasing depth: the extra parameters in MLP$(5g)$ improve generalization, while the extra layer in MLP(g, g) does not.